UTN FRLP

Desarrollo de Software 2026

React: Formularios y Validaciones

Clase #10 — 08 de junio del 2026

Matías Corti - Martina García Améndola

Temas

- El estado deja de ser un valor simple: listas y objetos que cambian
- Formularios controlados: capturar lo que escribe el usuario
- Validar los datos antes de guardarlos — y por qué importa
- Tres formas de validar, de menor a mayor: a mano → Zod → Zod + React Hook Form
- Redirigir después de una acción con useNavigate

De dónde venimos

La clase pasada vimos…

- Estructura por carpetas, Layout con children, React Router (Link, useParams)
- Inmutabilidad del estado — arrays (spread, map, filter) y objetos (spread)

Hoy sumamos:

- Esa misma inmutabilidad, ahora aplicada a formularios
- Inputs controlados (value + onChange)
- Validación: por qué se hace y cómo (con Zod)
- useNavigate para redirigir (se apoya en el Router de la clase pasada)



Antes de arrancar: dependencias

El aula no tiene internet confiable.
Sin instalación = sin práctica.

```bash
npm install zod react-hook-form @hookform/resolvers
```

✔ Verificar con: `npm run dev`



De nuevo: inmutabilidad

¿Por qué importa?

React compara el estado por referencia: si le devolvés el mismo array/objeto, cree que no cambió nada y **no re-renderiza**

```
// Arrays → nunca mutar, siempre devolver uno nuevo
setLibros([...libros, nuevo]);                // agregar
setLibros(libros.filter(l => l.id !== id));   // quitar
setLibros(libros.map(l => l.id === id ? { ...l, precio: 9999 } : l));
// editar

// Objetos → copio todo y piso lo que cambia
setLibro({ ...libro, titulo: 'Nuevo título' });
```

❌ push / pop / splice → mutan

✅ spread / map / filter → devuelven nuevo



La lista de libros ahora es estado

¿Qué cambia?

En C09 el catálogo era un `const` fijo.

Para dar de alta un libro, la lista tiene que poder cambiar → es estado

```
const [libros, setLibros] =
useState<LibroCardProps[]>(librosIniciales);

function agregarLibro(nuevo: LibroCardProps) {
  setLibros([...libros, nuevo]);   // CREATE, sin mutar
}
```

CRUD: hoy hacemos ***Create*** (alta). ***Read*** ya lo hacíamos (`.map`). ***Update*** y ***Delete*** quedan como extra.

🔼

¿Dónde vive la lista?

El problema: Dos páginas necesitan la misma lista. El catálogo la muestra y el alta la modifica. ¿Dónde la ponemos?

La regla: El estado *vive* en el ancestro común (acá, App) y *baja* a cada página por props.

```tsx
// App.tsx — App tiene la lista Y la función para modificarla
function App() {
  const [libros, setLibros] = useState<LibroCardProps[]>(librosIniciales);
  const agregarLibro = (nuevo: LibroCardProps) => setLibros([...libros, nuevo]);

  return (
    <Layout>
      <Routes>
        {/* dato: para MOSTRAR */}
        <Route path="/catalogo"     element={<LibrosCatalogo libros={libros} />} />
        {/* función: para AVISAR */}
        <Route path="/libros/nuevo" element={<LibroNuevo onAgregar={agregarLibro} />} />
      </Routes>
    </Layout>
  );
}
```

Los datos *bajan* por props (libros); las funciones también *bajan* (onAgregar), para que el hijo le "avise" al padre

El formulario no toca la lista: llama a onAgregar(nuevo) y App se encarga de agregarlo



FORMULARIOS EN REACT



Componentes controlados

Arrancamos con **un solo campo** para entender el mecanismo; en la próxima lo ampliamos a todo el formulario

¿Qué es?

Un input donde el **valor lo guarda el estado de React**, no el DOM. React es la única fuente de verdad

```
const [titulo, setTitulo] = useState('');

<Form.Control
  value={titulo}                          // el valor LO LEE del estado
  onChange={e => setTitulo(e.target.value)} // cada tecla ESCRIBE en el estado
/>
```

¿Por qué hacerlo así?

- Podés leer el valor en cualquier momento (para validar, enviar, etc.)
- Podés resetear, deshabilitar el botón o transformar lo que se escribe
- Sin `value` + `onChange` → input "muerto": el dato lo tiene el navegador y nosotros no lo vemos



El formulario completo: un objeto

Mismo mecanismo, muchos campos. Cada input sigue siendo controlado (value + onChange); lo único que cambia es dónde se guarda: en vez de un useState por campo, agrupamos todos en **un objeto**.

```
const [form, setForm] = useState({ titulo: '', autor: '', precio: '' });

const handleChange = (e: React.ChangeEvent<HTMLInputElement>) => {
  const { name, value } = e.target;
  setForm({ ...form, [name]: value });   // ← inmutabilidad de objetos (C09)
};


<Form.Control name="titulo" value={form.titulo} onChange={handleChange} />
<Form.Control name="autor"  value={form.autor}  onChange={handleChange} />
```

- No reemplaza nada: cada input lee/escribe en su campo del objeto (form.titulo)
- Un solo handleChange para todos, gracias al name + la clave dinámica [name]
- ¿Un useState por campo? Con 10 campos, inmanejable. Por eso, un objeto.

🔧 Distintos tipos de input

react-bootstrap nos da el componente para cada caso

```
<Form.Control name="titulo" />                {/* texto (type por defecto) */}

<Form.Control name="precio" type="number" … />          {/* número */}

<Form.Control as="textarea" name="resena" … />          {/* multilínea */}

<Form.Select name="genero" … >                          {/* opciones */}
  <option value="">Elegí…</option>
  <option value="novela">Novela</option>
</Form.Select>

<Form.Check label="Disponible" name="disponible"        {/* sí / no */}
  checked={form.disponible} onChange={handleCheck} />
```

- Texto es el `type` por defecto → `type="text"` es opcional
- ⚠️ El `checkbox` es el distinto: lee `e.target.checked` (booleano), no `e.target.value`



useNavigate: redirigir por código

¿Qué es?

El "primo" de `Link`: en vez de que el usuario haga click, redirigimos nosotros desde el código

```
import { useNavigate } from 'react-router-dom';
const navigate = useNavigate();

// después de un alta exitosa:
agregarLibro(nuevo);
navigate('/catalogo');   // lo llevamos al listado, sin recargar
```

Caso típico: termina el alta → lo mando al catálogo a ver su libro

Necesita el BrowserRouter configurado



VALIDACIÓN EN FRONTEND



¿POR QUÉ VALIDAR?

Validar es comprobar que los datos cumplen las reglas antes de usarlos o guardarlos

¿Por qué hacerlo en Frontend?

- **UX**: el usuario recibe feedback al instante y corrige sin esperar al servidor
- **Integridad de datos**: no entran precios negativos, campos vacíos ni formatos rotos
- **Menos bugs**: el resto del código puede asumir que los datos están sanos
- **Seguridad**: nunca confiar en lo que manda el usuario

⚠️ **Clave**: la validación del frontend es para la experiencia; la que protege de verdad es la del backend. En una app seria se valida en los dos lados



¿Por qué Validar en el Frontend?

❌ **Sin Validación**	✅ **Con Validación**
Usuario llena formulario ⏱️ 2s	Usuario llena formulario ⚡0s
Click en *Enviar* ➡️	Feedback **instantáneo** ✓
Espera respuesta del servidor ⏳	Corrige **mientras escribe** ✍️
Error devuelto por backend ❌	Solo se envía si es válido 👍
Vuelve a intentar 😤	Usuario feliz 😊

⚠️ ***IMPORTANTE:*** *La validación en frontend* ***mejora la experiencia de usuario (UX)****, pero* ***NO es seguridad.*** *Cualquiera puede saltarla con las DevTools.*



Validar a mano

Antes de guardar, comprobamos los datos.
La forma más directa: con `if`

```
const validar = () => {
  const errores: Record<string, string> = {};
  if (!form.titulo.trim()) errores.titulo = 'El título es obligatorio';
  if (!form.autor.trim())  errores.autor  = 'El autor es obligatorio';
  if (Number(form.precio) <= 0) errores.precio = 'Precio inválido';
  return errores;  // si está vacío, no hay errores
};
```

Funciona, pero...

- Se **repite** y se mezcla con la UI: "if espagueti" en cada formulario
- No hay un tipo que represente "un libro válido"
- Difícil de mantener cuando crecen las reglas

🎨 Mostrar los errores

react-bootstrap ya trae el patrón de error de formulario

El objeto errores de la slide anterior (`{ titulo: '...', precio: '...' }`) lo lee el formulario campo por campo:

```
<Form.Group className="mb-3">
  <Form.Label>Título</Form.Label>
  <Form.Control name="titulo" value={form.titulo} onChange={handleChange}
    isInvalid={!!errores.titulo} />        {/* ¿hay error en titulo? */}
  <Form.Control.Feedback type="invalid">
    {errores.titulo}                       {/* el mensaje de ese campo */}
  </Form.Control.Feedback>
</Form.Group>
```

- Si `errores.titulo` existe → `isInvalid` pinta el borde rojo y `Feedback` muestra el texto
- Si no existe (undefined) → el campo se ve normal
- Mismo patrón por campo: `errores.autor`, `errores.precio`, etc



Nivel 1 Consigna ~30 min

🎯 **Objetivo:**

Alta de libro controlada + validación a mano

Paso a paso

1. En `App.tsx`: pasar el catálogo a estado (`useState`) + función `agregarLibro`
2. Crear `pages/LibroNuevo.tsx`: formulario controlado (objeto-estado) con titulo, autor, precio, disponible
3. Ruta `/libros/nuevo` + link en el Header
4. Validar con `if` al submit; mostrar el error de cada campo
5. Si pasa: `agregarLibro(...)` redirigir al catálogo: `navigate('/catalogo')`

✅ **Si cargás un libro y aparece en el catálogo, nivel completado!**

```tsx
// pages/LibroNuevo.tsx
import { useState } from 'react';
import { useNavigate } from 'react-router-dom';
import { Form, Button } from 'react-bootstrap';
import type LibroCardProps from '../types/libroCardProps';

const IMG_PLACEHOLDER = 'https://placehold.co/300x400?text=Libro';
interface Props { onAgregar: (libro: LibroCardProps) => void; }

function LibroNuevo({ onAgregar }: Props) {
  const navigate = useNavigate();
  const [form, setForm] = useState({ titulo: '', autor: '', precio: '', disponible: true });
  const [errores, setErrores] = useState<Record<string, string>>({});

  const handleChange = (e: React.ChangeEvent<HTMLInputElement>) => {
    const { name, value, type, checked } = e.target;
    setForm({ ...form, [name]: type === 'checkbox' ? checked : value });
  };

  const validar = () => {
    const err: Record<string, string> = {};
    if (!form.titulo.trim()) err.titulo = 'El título es obligatorio';
    if (!form.autor.trim())  err.autor  = 'El autor es obligatorio';
    if (form.precio === '' || Number(form.precio) <= 0) err.precio = 'El precio debe ser mayor a 0';
    return err;
  };

  const handleSubmit = (e: React.FormEvent) => {
    e.preventDefault();
    const err = validar();
    if (Object.keys(err).length > 0) { setErrores(err); return; }
    onAgregar({
      id: Date.now(), titulo: form.titulo, autor: form.autor,
      precio: Number(form.precio), imagen: IMG_PLACEHOLDER, disponible: form.disponible,
    });
    navigate('/catalogo');
  };

  return (
    <Form onSubmit={handleSubmit} className="container py-4" style={{ maxWidth: 480 }}>
      <h2>Nuevo libro</h2>
      <Form.Group className="mb-3">
        <Form.Label>Título</Form.Label>
        <Form.Control name="titulo" value={form.titulo} onChange={handleChange} isInvalid={!!errores.titulo} />
        <Form.Control.Feedback type="invalid">{errores.titulo}</Form.Control.Feedback>
      </Form.Group>
      <Form.Group className="mb-3">
        <Form.Label>Autor</Form.Label>
        <Form.Control name="autor" value={form.autor} onChange={handleChange} isInvalid={!!errores.autor} />
        <Form.Control.Feedback type="invalid">{errores.autor}</Form.Control.Feedback>
      </Form.Group>
      <Form.Group className="mb-3">
        <Form.Label>Precio</Form.Label>
        <Form.Control type="number" name="precio" value={form.precio} onChange={handleChange} isInvalid={!!errores.precio} />
        <Form.Control.Feedback type="invalid">{errores.precio}</Form.Control.Feedback>
      </Form.Group>
      <Form.Check className="mb-3" label="Disponible" name="disponible" checked={form.disponible} onChange={handleChange} />
      <Button type="submit">Agregar libro</Button>
    </Form>
  );
}

export default LibroNuevo;
```



ZOD: VALIDACIÓN DECLARATIVA



¿Qué es Zod?

[ZOD: Documentación]

¿Qué es?

Una librería **para describir la forma de los datos** (un *schema*) y validarlos contra ella

```
import { z } from 'zod';

export const libroSchema = z.object({
 titulo: z.string().trim().min(1, 'El título es obligatorio'),
 autor:  z.string().trim().min(1, 'El autor es obligatorio'),
 precio: z.coerce.number().positive('El precio debe ser mayor a 0'),
 disponible: z.boolean(),
});

export type LibroValidado = z.infer<typeof libroSchema>;
                                    // el tipo TS sale del schema
```

¿Por qué conviene?

- Las reglas en un solo lugar, no repartidas en `if`
- `z.infer` te da el tipo TypeScript gratis (no lo escribís dos veces)
- El mismo schema se reusa en el backend



PARSE VS SAFEPARSE

	parse(datos)	safeParse(datos)
Si falla	lanza una excepción	devuelve `{ success: false, error }`
Si pasa	devuelve los datos	devuelve `{ success: true, data }`
Ideal para	backend (cortar todo)	frontend (mostrar errores sin romper)

Regla práctica:

- `safeParse` en el front
- `parse` en el back

🎯 safeParse → errores por campo

```ts
const resultado = libroSchema.safeParse(form);

if (!resultado.success) {
  const errores: Record<string, string> = {};
  for (const issue of resultado.error.issues) {
    // cada issue = { path, message }
    const campo = String(issue.path[0]);
    if (!errores[campo]) errores[campo] = issue.message;
  }
  setErrores(errores);
  return;
}
// resultado.data ya está validado y TIPADO como LibroValidado
agregarLibro({ ...resultado.data, id: Date.now(), imagen: PLACEHOLDER });
```

- El error trae `issues`: un `path` (qué campo) y un `message` (qué pasó)
- Lo recorremos y armamos el mismo `Record<campo, mensaje>` de antes
- Atajo: `z.flattenError(resultado.error).fieldErrors`



Nivel 2 Consigna ~20 min

🎯 **Objetivo:**

Misma alta, ahora validada con Zod

Paso a paso

1. Crear `schemas/libroSchema.ts` con `libroSchema` + `type LibroValidado = z.infer<...>`
2. Reemplazar los `if` por `libroSchema.safeParse(form)`
3. Armar los errores recorriendo `resultado.error.issues`
4. Mostrarlos con `isInvalid` + `Form.Control.Feedback` (que ya estaban)

✅ **Si los errores salen del schema y no de tus if, nivel completado!**



REACT HOOK FORM + ZOD



¿Qué es React Hook Form?

¿Qué es?

Una librería que se encarga del trabajo repetitivo de un formulario: el estado, el submit y los errores

¿Qué nos saca de encima?

- El `useState` por campo y el `handleChange`
- El armado manual del objeto de errores
- Optimiza los re-renders (más performante en forms grandes)

¿Cuándo conviene?

Cuando los formularios crecen y el armado a mano se vuelve repetitivo. Lo trajimos después de hacerlo a mano: así se ve qué problema resuelve

En el TP: controlado+Zod o RHF. Lo que sí o sí deben dominar es controlado + Zod



RHF + Zod: el mismo schema

```tsx
import { useForm } from 'react-hook-form';
import { zodResolver } from '@hookform/resolvers/zod';

const { register, handleSubmit, formState: { errors } } =
 useForm<LibroValidado>({ resolver: zodResolver(libroSchema) });
 // mismo schema

<Form onSubmit={handleSubmit(onSubmit)}>
  <Form.Control {...register('titulo')} isInvalid={!!errors.titulo} />
  <Form.Control.Feedback type="invalid">
    {errors.titulo?.message}
  </Form.Control.Feedback>
</Form>
```

- Sin `useState` por campo, sin `handleChange`, sin armar errores a mano
- `{...register('campo')}` funciona con `Form.Control` porque reenvía la `ref`



Nivel 3 Consigna ~25 min

🎯 **Objetivo:**

Misma alta, ahora validada con Zod

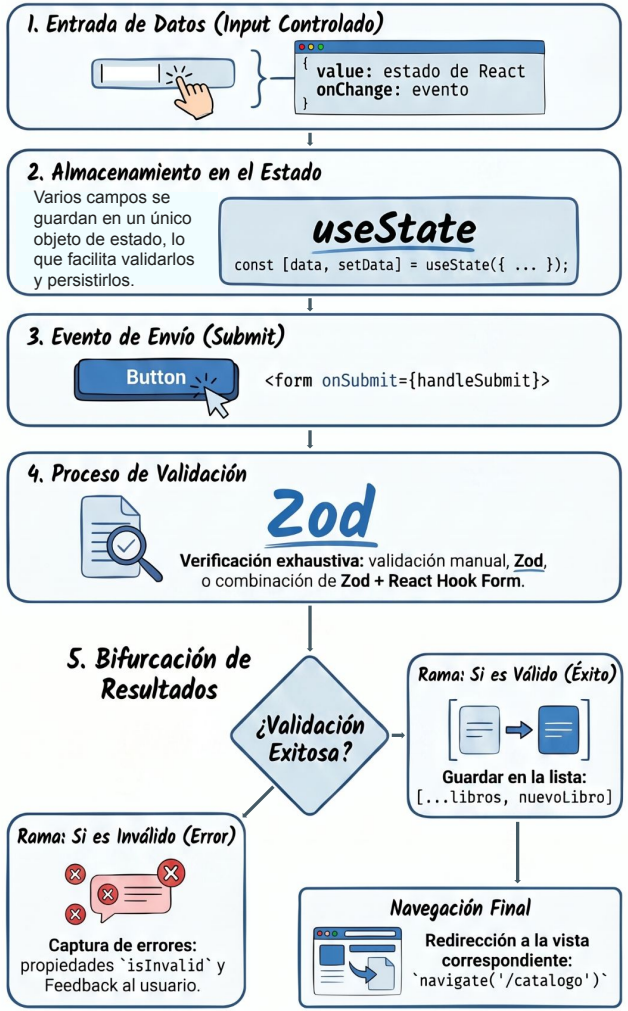
Paso a paso

1. `npm install react-hook-form @hookform/resolvers` (si falta)
2. `useForm<LibroValidado>({ resolver: zodResolver(libroSchema) })` ← ¡el mismo schema!
3. Cambiar los inputs a `{...register('campo')}` + `isInvalid={!!errors.campo}`
4. `onSubmit` con `handleSubmit` → `agregarLibro` + `navigate`

✅ Si quitaste los `useState`/`handleChange` y sigue andando, nivel completado!

El flujo completo

- Cambia la herramienta, **no el flujo:** capturar → validar → guardar+redirigir / mostrar
- A mano → Zod → Zod+RHF: mismo resultado, menos código





Recursos y referencias

- **Zod** (Basic usage, safeParse, Error formatting)
- **React Hook Form** (Get Started, useForm, resolvers)
- **react-bootstrap Forms**
- **DS26-Libreria C10** Repositorio de Ejemplo

Recordá: la doc oficial es la fuente principal

Actividades

Actividad Individual de la semana

1° instancia hasta el **21/06/2026**
2° instancia: ***05/07/2026***
(Cierre Bloque 2)

[AI-C10] Alta de libro con validación

Sumar Layout y Navegación a la librería React

- En el repositorio ds2026-actividades copiar el sitio de la librería de la Clase 09 en subdirectorio nuevo para esta clase: **ejercicios/c10-formularios-zod/**
- Crear el alta de libro con validaciones visto en la clase.

Subir al repositorio y comunicar la finalización en Discord en un mensaje dentro del canal **#actividades**, en el Hilo: "***[AI-C09] Alta de libro con validación***".

Actividad grupal

[AG-C10 - Formulario + validaciones en el TP]

1. Sumar al menos un formulario de alta real del dominio del TP
2. Validar con Zod (controlado+Zod obligatorio; RHF recomendado)
3. Mostrar errores por campo en la UI (react-bootstrap)
4. Redirigir tras el alta con useNavigate
5. Dividir las tareas en el equipo
6. Hacer PRs cruzados (al menos uno cada alumno)
7. Trello: mover tarjetas correspondientes a "Done"
8. Comunicar la finalización en Discord en un mensaje único por equipo, dentro del canal **#actividades**, en el Hilo: "***[AG-C010] Formulario + validaciones del TP***".

Hasta el **21/06/2026** (inclusive)